

PreExam Solution

1 Ejercicio 1: Inferencia de Tipos

Se analiza la función:

```
(define (count x l)
  (if (null? l)
      0
      (if (equal? (car l) x)
          (+ 1 (count x (cdr l)))
          (count x (cdr l)))))
```

Paso 1: Identificación de variables

- x : tipo desconocido $\Rightarrow T$
- l : lista de elementos del mismo tipo que $x \Rightarrow T \text{ list}$

Paso 2: Análisis de operaciones

- `(null? l)` requiere $l : T \text{ list}$ y devuelve **bool**
- `(car l)` devuelve el primer elemento $\Rightarrow T$
- `(cdr l)` devuelve la cola $\Rightarrow T \text{ list}$
- `equal?(car(l), x)` requiere ambos del mismo tipo $\Rightarrow$ ambos T , confirmando $x : T$

Paso 3: Tipo de retorno

La función retorna:

- 0 en el caso base $\Rightarrow$ **int**
- `(+ 1 (count ...))` en el caso recursivo $\Rightarrow$ **int**

Ambas ramas coinciden: el tipo de retorno es **int**.

Resultado final

$count : T \rightarrow T \text{ list} \rightarrow \text{int}$

Patrón clave: funciones recursivas sobre listas suelen tener tipo $T \rightarrow T \text{ list} \rightarrow \alpha$. El tipo es *polimórfico* en T : funciona para listas de enteros, strings, etc.

2 Ejercicio 2: Semántica Operacional

Se desea definir la semántica de la instrucción:

repeat E **times** C

Idea clave

- La expresión E se evalúa **una sola vez** al inicio.
- El resultado n es el contador; se decrementa en 1 después de cada ejecución de C .
- Introducimos una forma auxiliar **repeat-n** k **times** C donde $k \in \mathbb{N}$ ya es un valor entero (no una expresión).

Regla de entrada (evalúa E una sola vez)

$$\frac{\langle E, \sigma \rangle \Downarrow n \quad \langle \text{repeat-n } n \text{ times } C, \sigma \rangle \Downarrow \sigma'}{\langle \text{repeat } E \text{ times } C, \sigma \rangle \Downarrow \sigma'} \quad [\text{REPEAT-EVAL}]$$

Caso base ($k = 0$)

$$\overline{\langle \text{repeat-n } 0 \text{ times } C, \sigma \rangle \Downarrow \sigma} \quad [\text{REPEAT-ZERO}]$$

Caso recursivo ($k > 0$)

$$\frac{k > 0 \quad \langle C, \sigma \rangle \Downarrow \sigma' \quad \langle \text{repeat-n } (k-1) \text{ times } C, \sigma' \rangle \Downarrow \sigma''}{\langle \text{repeat-n } k \text{ times } C, \sigma \rangle \Downarrow \sigma''} \quad [\text{REPEAT-STEP}]$$

Corrección importante: en la regla REPEAT-STEP, *no* se vuelve a evaluar E . Solo se trabaja con el contador entero k y se resta 1. Re-evaluar E en cada iteración violaría la especificación del enunciado.

Intuición: equivalente a un **for-loop** donde el límite superior se fija antes de entrar al ciclo.

3 Ejercicio 3: Traducción Infix a Postfix

Función semántica

Definimos $\mathcal{P}[\cdot] : \mathbf{Expr} \rightarrow \mathbf{String}$ que traduce una expresión infija a postfija.

Reglas denotacionales

$$\begin{aligned} \mathcal{P}[n] &= \text{"n"} \\ \mathcal{P}[E_1 + E_2] &= \mathcal{P}[E_1] \frown \mathcal{P}[E_2] \frown \text{"+"} \\ \mathcal{P}[E_1 - E_2] &= \mathcal{P}[E_1] \frown \mathcal{P}[E_2] \frown \text{"-"} \\ \mathcal{P}[E_1 * E_2] &= \mathcal{P}[E_1] \frown \mathcal{P}[E_2] \frown \text{"*"} \\ \mathcal{P}[E_1 / E_2] &= \mathcal{P}[E_1] \frown \mathcal{P}[E_2] \frown \text{" / " } \end{aligned}$$

donde $\frown$ denota concatenación de strings (con espacio separador).

Ejemplo: $3 + 4 * 5$

Con precedencia estándar ($*$ liga más que $+$), el árbol es $3 + (4 * 5)$:

$$\begin{aligned}\mathcal{P}[\![3 + (4 * 5)]\!] &= \mathcal{P}[\![3]\!] \frown \mathcal{P}[\![4 * 5]\!] \frown "+" \\ &= "3" \frown (\mathcal{P}[\![4]\!] \frown \mathcal{P}[\![5]\!] \frown "*") \frown "+" \\ &= "3" \frown "4" \frown "5" \frown "*" \frown "+" \\ &= "3\ 4\ 5\ *\ +"\end{aligned}$$

Regla clave: izquierdo $\frown$ derecho $\frown$ operador. La precedencia la maneja el *parser* al construir el AST; la función $\mathcal{P}$ solo recorre el árbol resultante.

4 Ejercicio 4: Alcance Estático vs. Dinámico

Los dos modelos

Modelo	Pregunta a hacer	Resultado
Estático (léxico)	¿Dónde está <i>definido</i> <code>proc1</code> en el fuente?	$x = 2$
Dinámico	¿Qué hay en la <i>pila de llamadas</i> ahora mismo?	$x = 3$

Alcance estático

`proc1` está definido en el **nivel global** del programa. El único x visible en su contexto de definición es el x global, cuyo valor es 2 (asignado por `main` antes de llamar a `proc2`).

$\Rightarrow$ `print` imprime **x=2**

Alcance dinámico

En tiempo de ejecución, la pila al momento del `print` es:

$$\underbrace{\text{main}}_{x=2} \longrightarrow \underbrace{\text{proc2}}_{x=3 \text{ (local)}} \longrightarrow \text{proc1}$$

El enlace más reciente de x en la pila es el de `proc2`, con valor 3.

$\Rightarrow$ `print` imprime **x=3**

Regla mental:

Estático = dónde se *define* (contexto léxico)

Dinámico = desde dónde se *llama* (pila en tiempo de ejecución)

5 Ejercicio 5: Paso de Parámetros

Tipos de parámetros

- **ref** (por referencia): alias directo al argumento. Leer/escribir el parámetro lee/escribe la variable original.
- **name** (por nombre / Jensen's device): la expresión argumento se *re-evalúa* cada vez que se usa el parámetro. Si las variables internas cambian, cada uso puede acceder a una celda diferente.
- **VR** (value-result): copia *in* al entrar ($z_{\text{local}} \leftarrow$ valor actual del argumento) y copia *out* al salir ($i \leftarrow z_{\text{local}}$).

Estado inicial

$$a = \{0, 10, 20, 40\}, \quad i = 0$$

Llamada: `test(i, a[i], i)`

- $x \xleftarrow{\text{ref}} i$: alias de i
- $y \xleftarrow{\text{name}} a[i]$: expresión diferida
- $z_{\text{local}} \xleftarrow{\text{VR}} 0$ (copia-in de $i = 0$)

Ejecución línea a línea

Línea	Sentencia	Efecto	Estado tras ejecutar
8	<code>x := x+1</code>	x es alias de i ; $i \leftarrow 0+1 = 1$	$i = 1, z_{\text{local}} = 0$
9	<code>y := y+1</code>	$y = a[i]; i = 1$ ahora $\Rightarrow y = a[1] = 10$	$a[1] \leftarrow 11$
10	<code>z := z+2</code>	copia local $z_{\text{local}} \leftarrow 0+2 = 2$	$z_{\text{local}} = 2, i = 1$
11	<code>y := y+1</code>	$y = a[i]; i = 1$ todavía $\Rightarrow y = a[1] = 11$	$a[1] \leftarrow 12$
Retorno (VR copia-out)		$z_{\text{local}} = 2 \Rightarrow i \leftarrow 2$	$i = 2$

Resultado final

$$i = 2, \quad a = \{0, 12, 20, 40\}$$

Tres interacciones críticas:

1. **ref cambia el índice de name.** La línea 8 incrementa i de 0 a 1 vía **ref**. Las líneas 9 y 11 usan $y = a[i]$ con $i = 1$, accediendo a $a[1]$ (no $a[0]$).
2. **name modifica la misma celda dos veces.** Tanto la línea 9 como la 11 operan sobre $a[1]$, incrementándolo de 10 a 11 y luego a 12.
3. **VR sobrescribe el cambio de ref al retornar.** **ref** dejó $i = 1$, pero la copia-out de VR escribe $z_{\text{local}} = 2$ sobre i , resultando en $i = 2$.

5.1 Worked Examples

The following examples are designed to address two common sources of confusion: the difference between big-step and small-step style, and how to extend the semantics with new constructs. Each example is worked out in full.

5.1.1 Example 1: Big-step vs. Small-step on the Same Program

Consider the following SIMP program with initial state $\sigma = \{x \mapsto 3, y \mapsto 0\}$:

```
y := !x + 1 ;
x := !y * 2
```

Big-step derivation. Big-step semantics derives the *final* state in one tree. We write $\langle C, \sigma \rangle \Downarrow \sigma'$ to mean “command C in state σ terminates in state σ' ”.

The key insight: in big-step style you build a **derivation tree** the root is the whole program, the leaves are axioms. Work *top-down* by applying the sequence rule, then fill in each sub-derivation.

$$\frac{\frac{\langle !x + 1, \sigma \rangle \Downarrow 4}{\langle y := !x + 1, \sigma \rangle \Downarrow \sigma_1} \quad \frac{\langle !y * 2, \sigma_1 \rangle \Downarrow 8}{\langle x := !y * 2, \sigma_1 \rangle \Downarrow \sigma_2}}{\langle y := !x + 1 ; x := !y * 2, \sigma \rangle \Downarrow \sigma_2}$$

where $\sigma_1 = \sigma[y \mapsto 4]$ and $\sigma_2 = \sigma_1[x \mapsto 8]$.

Final state: $\sigma_2 = \{x \mapsto 8, y \mapsto 4\}$.

Small-step derivation. Small-step semantics reduces the program *one step at a time*. We write $\langle C, \sigma \rangle \rightarrow \langle C', \sigma' \rangle$ (or $\langle C, \sigma \rangle \rightarrow \langle \text{skip}, \sigma' \rangle$ at the last step).

The key insight: in small-step style you write a **linear sequence** of configurations. At each step, apply exactly one rule to the leftmost reducible sub-expression.

$$\begin{array}{ll}
\langle y := !x + 1 ; x := !y * 2, \sigma \rangle & \\
\rightarrow \langle y := 3 + 1 ; x := !y * 2, \sigma \rangle & \text{(dereference !x)} \\
\rightarrow \langle y := 4 ; x := !y * 2, \sigma \rangle & \text{(arithmetic)} \\
\rightarrow \langle \mathbf{skip} ; x := !y * 2, \sigma_1 \rangle & \text{(assign, } \sigma_1 = \sigma[y \mapsto 4]) \\
\rightarrow \langle x := !y * 2, \sigma_1 \rangle & \text{(skip-seq)} \\
\rightarrow \langle x := 4 * 2, \sigma_1 \rangle & \text{(dereference !y)} \\
\rightarrow \langle x := 8, \sigma_1 \rangle & \text{(arithmetic)} \\
\rightarrow \langle \mathbf{skip}, \sigma_2 \rangle & \text{(assign, } \sigma_2 = \sigma_1[x \mapsto 8])
\end{array}$$

Both styles reach the same final state $\{x \mapsto 8, y \mapsto 4\}$.

When to use each.

- Use **big-step** when you only care about the final result and want a compact derivation.
- Use **small-step** when you need to reason about intermediate states, non-termination, or concurrency.
- Never mix styles in the same derivation.

5.1.2 Example 2: Extending SIMP with for

We extend SIMP with the construct **for** E_1 **to** E_2 **do** C , which evaluates E_1 and E_2 *once* at the start and executes C for each integer i with $n_1 \leq i \leq n_2$ (does nothing if $n_1 > n_2$).

Step 1 Define the rules The standard strategy is to *desugar* the new construct into existing ones once the bounds are known. We need three rules.

Evaluate the bounds (two expression-reduction steps, small-step):

$$\begin{array}{c}
\frac{\langle E_1, \sigma \rangle \rightarrow \langle E'_1, \sigma \rangle}{\langle \mathbf{for } E_1 \text{ to } E_2 \text{ do } C, \sigma \rangle \rightarrow \langle \mathbf{for } E'_1 \text{ to } E_2 \text{ do } C, \sigma \rangle} \\
\frac{\langle E_2, \sigma \rangle \rightarrow \langle E'_2, \sigma \rangle}{\langle \mathbf{for } n_1 \text{ to } E_2 \text{ do } C, \sigma \rangle \rightarrow \langle \mathbf{for } n_1 \text{ to } E'_2 \text{ do } C, \sigma \rangle}
\end{array}$$

Base case bounds are values, loop is empty ($n_1 > n_2$):

$$\frac{n_1 > n_2}{\langle \mathbf{for } n_1 \text{ to } n_2 \text{ do } C, \sigma \rangle \rightarrow \langle \mathbf{skip}, \sigma \rangle}$$

Inductive case unfold one iteration ($n_1 \leq n_2$):

$$\frac{n_1 \leq n_2}{\langle \mathbf{for } n_1 \text{ to } n_2 \text{ do } C, \sigma \rangle \rightarrow \langle C ; \mathbf{for } (n_1+1) \text{ to } n_2 \text{ do } C, \sigma \rangle}$$

Step 2 Trace through an example. Use state $\sigma = \{x \mapsto 0, n_1 \mapsto 1, n_2 \mapsto 3\}$ and program `for ! $n_1$  to ! $n_2$  do  $x := !x + 1$ .`

$\langle \text{for } !n_1 \text{ to } !n_2 \text{ do } x := !x + 1, \sigma \rangle$	
$\rightarrow \langle \text{for } 1 \text{ to } !n_2 \text{ do } x := !x + 1, \sigma \rangle$	(deref ! n_1)
$\rightarrow \langle \text{for } 1 \text{ to } 3 \text{ do } x := !x + 1, \sigma \rangle$	(deref ! n_2)
$\rightarrow \langle x := !x + 1 ; \text{for } 2 \text{ to } 3 \text{ do } x := !x + 1, \sigma \rangle$	(unfold, $1 \leq 3$)
$\rightarrow \langle \text{skip} ; \text{for } 2 \text{ to } 3 \text{ do } x := !x + 1, \sigma_1 \rangle$	(assign, $\sigma_1 = \sigma[x \mapsto 1]$)
$\rightarrow \langle \text{for } 2 \text{ to } 3 \text{ do } x := !x + 1, \sigma_1 \rangle$	(skip-seq)
$\vdots$	
$\rightarrow \langle \text{skip}, \sigma_3 \rangle$	(base case, $4 > 3, \sigma_3 = \sigma[x \mapsto 3]$)

Common mistakes to avoid.

- **Evaluating the bounds repeatedly.** The bounds n_1 and n_2 must be fully reduced to values *before* unfolding begins. If you unfold first, you may re-evaluate a side-effectful expression on every iteration.
- **Forgetting the base case.** Without the $n_1 > n_2$ rule, the derivation never terminates for empty ranges.
- **Mixing big-step and small-step notation.** In big-step, the `for` rules look different, each premise is a full $\Downarrow$ judgment, not a single $\rightarrow$ step.